30/10/2025

TASK - 2.

REAL ESTATE HOUSING PRICE PREDICTION

- 20,640 rows - each row represents a housing district / block group.
- 10 columns - 9 numerical columns and 1 categorical column.

Step 1:- Library Import & Dataset Loading.

Code

```
import pandas as pd.          → handles datasets and tables.
import numpy as np            → performs numerical calculations.
import matplotlib.pyplot as plt → creates graph.
import seaborn as sns.        → creates statistical visualizations and heatmaps.
```

Loading dataset.

```
df = pd.read_csv('/content/housing2.csv')
df.head()
```

o/p:- Displays 1st 5 rows

Step 2:- EDA (Exploratory Data Analysis)

- Understand the dataset structure.
- Identify missing values
- Understand stat. info
- Identify unusual values.
- Discover relationships btw variables.

Code

o/p:-

df.shape → (20640, 10)

df.columns → col. names.

df.info → col. names, no. of non null values, data types, memory usage.

df.isnull().sum() → gives null values.

df.describe() → gives remaining stat values.

df['ocean_proximity'].value_counts()

↓

counts unique categorical value.

Columns names from dataset:-

[longitude, latitude, housing_median_age, total rooms, total_bedrooms, population, households, median_income, median_house_value, ocean_proximity.]

Step 3:- Data Preprocessing

Missing values :-

only 'Total_bedrooms' col has → 207 missing val.

replace with 'Median'

Why Median?

↓

is the middle value when data is arranged in ascending order.

It is less affected by extreme values than the mean.

Code

df['total_bedrooms'] = df['total_bedrooms'].fillna(df['total_bedrooms'].median())

df.isnull().sum()

↳ to chk if missing values have been handled.

o/p:-

Yes all col → 0 missing values

3.2 Separate Numerical and Categorical Columns.

Code

```
numerical_cols = df.select_dtypes(include=np.number).columns
categorical_cols = df.select_dtypes(include='object').columns
print("Numerical columns:", list(numerical_cols))
print("Categorical columns:", list(categorical_cols))
```

o/p:- Numerical col:- names
categorical col:- names.

3.3 Encode Categorical Data.

↓
converts categorical → numerical refer.

Eg:- ocean_proximity.

- INLAND
- NEAR BAY
- <1H OCEAN
- NEAR OCEAN
- ISLAND.

- Use One-Hot Encoding.

↓
A technique that converts each category into a separate binary column containing 0 or 1.

Categorical encoding:- Process of converting categorical → numerical data

Missing value imputation:- Process of replacing missing values with mean, median or mode.

Median imputation:- Replacing missing value with median of corresponding column.

Code

df = pd.get
df.head()
df.shape()

Step 4:

Intro

Creat

Cod

R
df['R

Be
df['

Cr
df[

Code

```
df = pd.get_dummies(df, columns=['ocean_proximity'], dtype=int)
df.head()
df.shape() → (20640, 14)
```

Step 4:- Feature Engineering.

↓

is the process of creating new features from existing features from existing features to help a machine learning model learn useful patterns and relationships.

Interaction Features

↓

are new features created by combining two or more existing features

Eg:- * Total rooms & households.
* Total bedrooms & total rooms.

Create New Features

Code

```
# Rooms per household
df['R rooms_per_household'] = (df['total rooms'] / df['households'])
# Bedrooms per room
df['bedrooms_per_room'] = (df['total_bedrooms'] / df['total_rooms'])
```

Create an Interaction Feature Code

```
df['income_rooms_interaction'] = (df['median_income'] * df['rooms_per_household'])
```

* This combines median income and rooms per household.

Display the New Features [Code]

df[['rooms_per_household', 'bedrooms_per_room', 'population_per_household', 'income_room_interaction']].head()

df.shape (20640, 14)

Steps:- Correlation Analysis.

↓

measures the strength and direction of the relationship btw two variables.

It value ranges from -1 to +1.

+1 - Perfect +ve correlation

0 - No linear "

-1 - Perfect -ve "

Correlation Matrix :-

table showing corr. btw numerical variables.

Multicollearity :-

condition where 2 or more independent var. are highly correlated.

Regularization :-

a technique used to control model complexity by penalizing large coeff.

Calculate the correlation Matrix

[Code]

→ calculates corr. btw num. col.

correlation = df.corr(numeric_only = True)

correlation.

↓ excludes non-numerical col.

stores the resulting correlation matrix.

Visualize

[Code]

import matpl

import seabo

plt.figure(

sns.heatma

plt.title(

plt.show()

Identif

[Cod

correla

Und

Why

Ho

Visualize Correlation Using a Heatmap

↓ represents cor. values using diff colour

[Code]

```
import matplotlib.pyplot as plt.
import seaborn as sns.
plt.figure(figsize=(14,10))  → adj graph size.
sns.heatmap(correlation, annot=True, cmap='coolwarm', fmt='.2f')
                                  ↳ set the color scheme
                          ↳ displays cor. values.
plt.title('correlation Heatmap')
plt.show()
```

Identify features Related to House Prices.

[Code]

```
correlation['median_house_value'].sort_values(ascending=False)
```

Understanding Multicollinearity.

Why is it a prob?

* Can make regression coeff unstable.
* difficult to identify individual effect of each feature.
* Can affect the interpretation of regression results.

Handle it.

- Ridge Regression (L2 Regularization)
- Lasso " (L1 ")

Step 6:- Train-Test Split and Standardization.

[Code]

```
x = df.drop('median_house_value', axis=1)   → removes the target col.
y = df['median_house_value']
```

x → contain Ip features / Indep. var.

y → target feature / dep. var.

Split the dataset

Code

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                   random_state = 42)
```

20% for testing

random_state = 42 → ensures reproducible splitting.

Check the shapes.

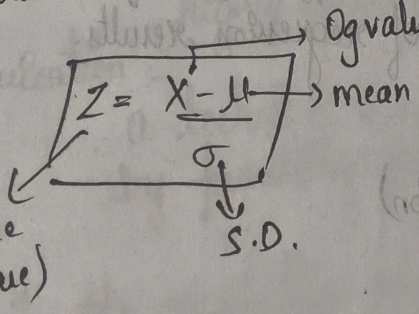
Code

```
print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

O/P:-
(16512, 13) → Training data
(4128, 13) → Testing data
(16512,)
(4128,)

Standardization

Process of transforming numerical features to have approx zero mean and unit std. dev.



* It transforms the data so that it has approx.

Mean = 0

S.D = 1

20% for testing
↑ testing
= 0.2

Code

```
from sklearn.preprocessing import StandardScaler.
categorical_features = [col for col in x.columns
import numpy as np.
scaler = StandardScaler()
x_train_scaled = x_train.copy()
x_test_scaled = x_test.copy()
continuous_features = x.select_dtypes(include=np.number).columns.tolist()
continuous_features = [col for col in continuous_features if not col.startswith
('ocean_proximity')]
x_train_scaled[continuous_features] = scaler.fit_transform(x_train[continuous_
                                                          features])
x_test_scaled[continuous_features] = scaler.transform(x_test[continuous_
                                                     features])
print("Training data shape:", x_train_scaled.shape) → (16512,13)
print("Testing data shape:", x_test_scaled.shape) → (4128,13)
```

Step 1:- Ordinary Least Squares (OLS) Regression

↓↓

is a linear regression technique that finds the best-fitting line by minimizing the sum of squared diff. btw actual & predicted value.

Mathematical Formula:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n$$

- Y ← Predicted house price.
- β_0 ← Intercept
- $\beta_1, \beta_2, \ldots$ ← Regr. coeff
- $X_1, X_2, \ldots, X_n$ ← I/P features.

Code

```
from sklearn.linear_model import LinearRegression
ols_model = LinearRegression()
ols_model.fit(X_train_scaled, y_train)
ols_prediction = ols_model.predict(X_test_scaled)
print("OLS Regression Model Trained Successfully!")
```

O/P ↑

Evaluate the OLS Model

MAE - Mean Absolute Error.
⇓
Avg abs. prediction error.

$$MAE = \frac{1}{n} \Sigma |y_i - \hat{y}_i|$$

RMSE - Root Mean Squared Error.
⇓
Penalizes larger prediction errors more heavily.

$$RMSE = \sqrt{\frac{1}{n} \Sigma (y_i - \hat{y}_i)^2}$$

R^2 Score - Coeff. of Determination.
⇓
Measures how much variation in house prices the model explains.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Code

```
from sklearn.metrics import mean_absolute_error, mean_squared_error,
    r2_score
ols_mae = mean_absolute_error(y_test, ols_predictions)
ols_rmse = np.sqrt(mean_squared_error(y_test, ols_predictions))
ols_r2 = r2_score(y_test, ols_predictions)
```

print("OLS MAE", ols_mae) ⇒ 5116.28262079208

print("OLS RMSE:", ols_rmse) ⇒ 72971.49471509317

print("OLS R2 score:", ols_r2) ⇒ 0.5935837724463086

Step 8: Ridge and Lasso Regression with GridSearchCV

hyperparameter tuning technique that test all diff parameter combinations using cross validation

Regularization

is a technique used in ML to prevent overfitting by adding a penalty to large model coeff.

Ridge Reg	Lasso Reg
Uses L2 regularization	L1.
Shrinks coeff. towards zero.	Can make some coeff exactly zero.
Generally retains all features.	Can perform feature selection.

Ridge Regression. (L2)

is a technique that adds a penalty based on the square of the coeff.

$$\text{Loss} = SSE = \alpha \sum \beta_i^2$$

- Loss ⇓ Sum of squared errors.
- α → regularization strength.
- β_i^2 → Reg. coeff.

Code

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
ridge = Ridge()
ridge_params = {'alpha': [0.01, 0.1, 1, 10, 100]}
```

```
ridge_grid = GridSearch(ridge, ridge_params, cv=5, scoring="neg_mean_
                                                squared_error")
ridge_grid.fit(X_train_scaled, y_train)
best_ridge = ridge_grid.best_estimator_
ridge_predictions = best_ridge.predict(X_test_scaled)
print("Best Ridge Alpha:", ridge_grid.best_params_)
print("Ridge Model Trained Successfully.")
```

O/P:-
{'alpha': 1}

Lasso Regression (L1)

⇓

Is a technique that adds a penalty based on the abs values of coeff.

$$Loss = SSE + \alpha \sum |\beta_i|$$

Code

```
from sklearn.linear_model import Lasso.
lasso = Lasso(max_iter=10000)
lasso_params = {'alpha': [0.01, 0.1, 1, 10, 100]}
lasso_grid = GridSearchCV(lasso, lasso_params, cv=5, scoring='neg_mean_
                                                squared_error')
lasso_grid.fit(X_train_scaled, y_train)
best_lasso = lasso_grid.best_estimator_
lasso_prediction = best_lasso.predict(X_test_scaled)
print("Best Lasso Alpha, lasso_grid.best_params_) => {'alpha': 1}
print("Lasso Model Trained Successfully!") <=> same.
```

O/P:-

Step 9:- Model Evaluation and Comparison.

→ diff. models (compare) using evaluation metrics to identify the most suitable model.

↓

process of measuring how accurately a trained ML model predicts unseen data.

- Lower MAE = better
- Lower RMSE = "
- Higher R^2 = "

Calculate Evaluation Metrics

Code

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score.
ols_mae = mean_absolute_error(y_test, ols_predictions)
ols_rmse = np.sqrt(mean_squared_error(y_test, ols_predictions))
ols_r2 = r2_score(y_test, ols_predictions)
ridge_mae = mean_absolute_error(y_test, ridge_predictions)
ridge_rmse = np.sqrt(mean_squared_error(y_test, ridge_predictions))
ridge_r2 = r2_score(y_test, ridge_predictions)
lasso_mae = mean_absolute_error(y_test, lasso_predictions)
lasso_rmse = np.sqrt(mean_squared_error(y_test, lasso_predictions))
lasso_r2 = r2_score(y_test, lasso_predictions)
print("OLS:", ols_mae, ols_rmse, ols_r2)
print("Ridge:" ridge_mae, ridge_rmse, ridge_r2)
print("Lasso:", lasso_mae, lasso_rmse, lasso_r2)
```

Create a Model Comparison Table

Code

```
comparison = pd.DataFrame({'Model': ['OLS', 'Ridge', 'Lasso'],
    'MAE': [ols_mae, ridge_mae, lasso_mae],
    'RMSE': [ols_rmse, ridge_rmse, lasso_rmse],
    'R2 Score': [ols_r2, ridge_r2, lasso_r2]})

comparison
```

Visualize Model Comparison.

Code

```
plt.figure(figsize=(8,5))
plt.bar(comparison['Model'], comparison['R2 Score'])
plt.title('R2 Score Comparison')
plt.xlabel('Regression Models')
plt.ylabel('R2Score')
plt.show()
```

Step 10:- Visualization.

Actual Values: Og target values present in the testing dataset.

Predicted values: values estimated by a trained ML model.

Code

```
plt.figure(figsize=(10,6))
plt.scatter(y_test, ols_predictions, label='OLS', alpha=0.4)
plt.scatter(y_test, ridge_predictions, label='Ridge', alpha=0.4)
plt.scatter(y_test, lasso_predictions, label='Lasso', alpha=0.4)
minimum = min(y_test.min(), ols_prediction.min(), ridge_predictions.min(),
              lasso_predictions.min())
```

plt.plot([minimum,
plt.xlabel('Actual
plt.ylabel('Predict
plt.title('Actual
plt.legend()
plt.show()

Residual

Residu

Ideally:

Code

ols_resid
ridge_re
lasso_re
plt.figu
plt.hist
plt.his
plt.his
plt.xl
plt.yl
plt.t
plt.
plt.

```
maximum = max(y_test.max(), ols_prediction.max(), ridge_predictions.max(),
                                                    lasso_predictions.max())
plt.plot([minimum, maximum], [minimum, maximum], 'r--', label='Perfect Prediction')
plt.xlabel('Actual House Prices')
plt.ylabel('Predicted House Prices')
plt.title('Actual vs Predicted House Prices')
plt.legend()
plt.show()
```

Residual Error Distribution → graphical rep. showing the frequency and spread of prediction errors.

Residual = Actual - Prediction

Ideally:
- Residuals should be centred around 0.
- Smaller errors are preferable.
- Large errors should be relatively uncommon.

Code

```
ols_residuals = y_test - ols_predictions
ridge_residuals = y_test - ridge_predictions
lasso_residuals = y_test - lasso_predictions
plt.figure(figsize=(10,6))
plt.hist(ols_residuals, bins=50, alpha=0.5, label='OLS')
plt.hist(ridge_residuals, bins=50, alpha=0.5, label='Ridge')
plt.hist(lasso_residuals, bins=50, alpha=0.5, label='Lasso')
plt.xlabel('Residual Error')
plt.ylabel('Frequency')
plt.title('Residual Error Distribution')
plt.legend()
plt.show()
```